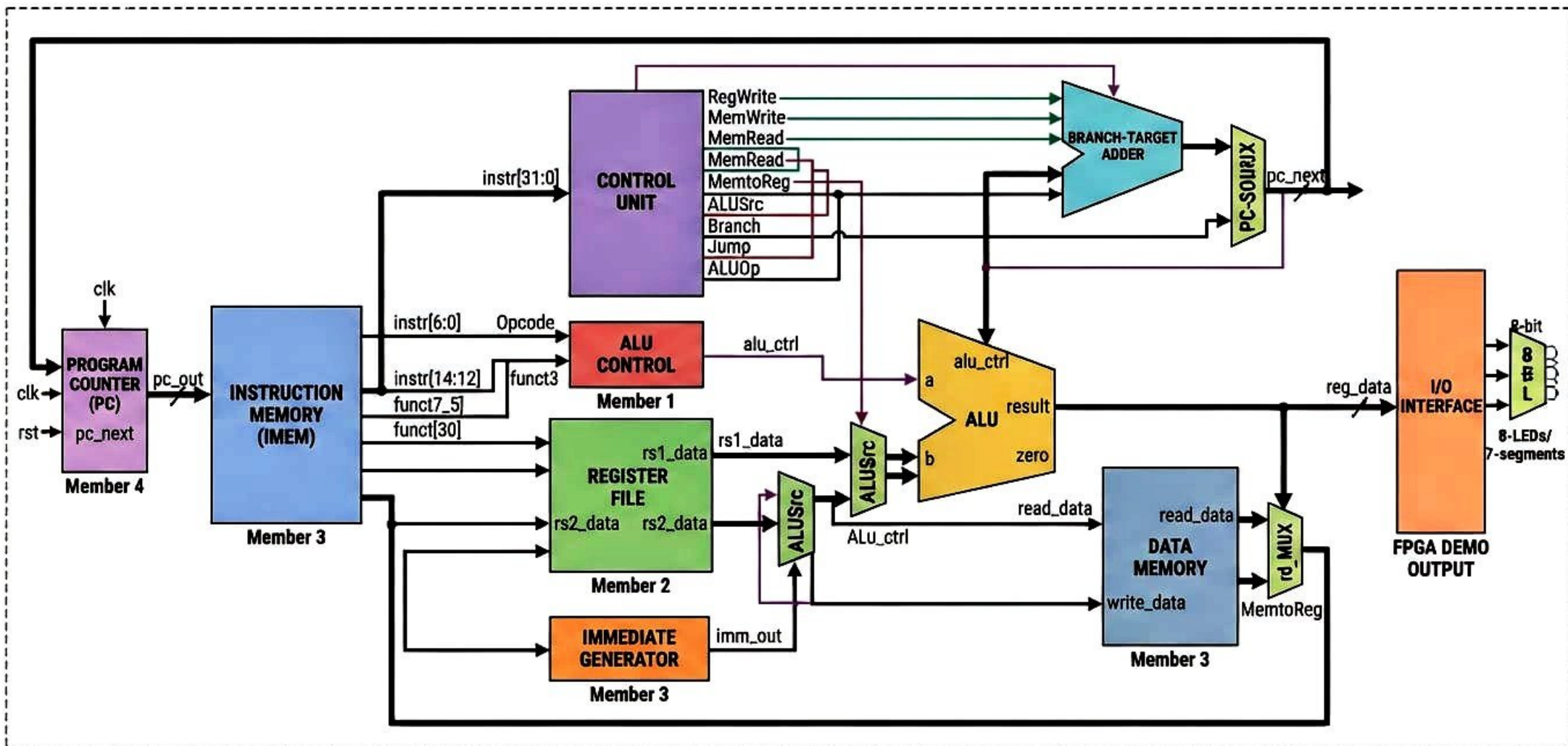
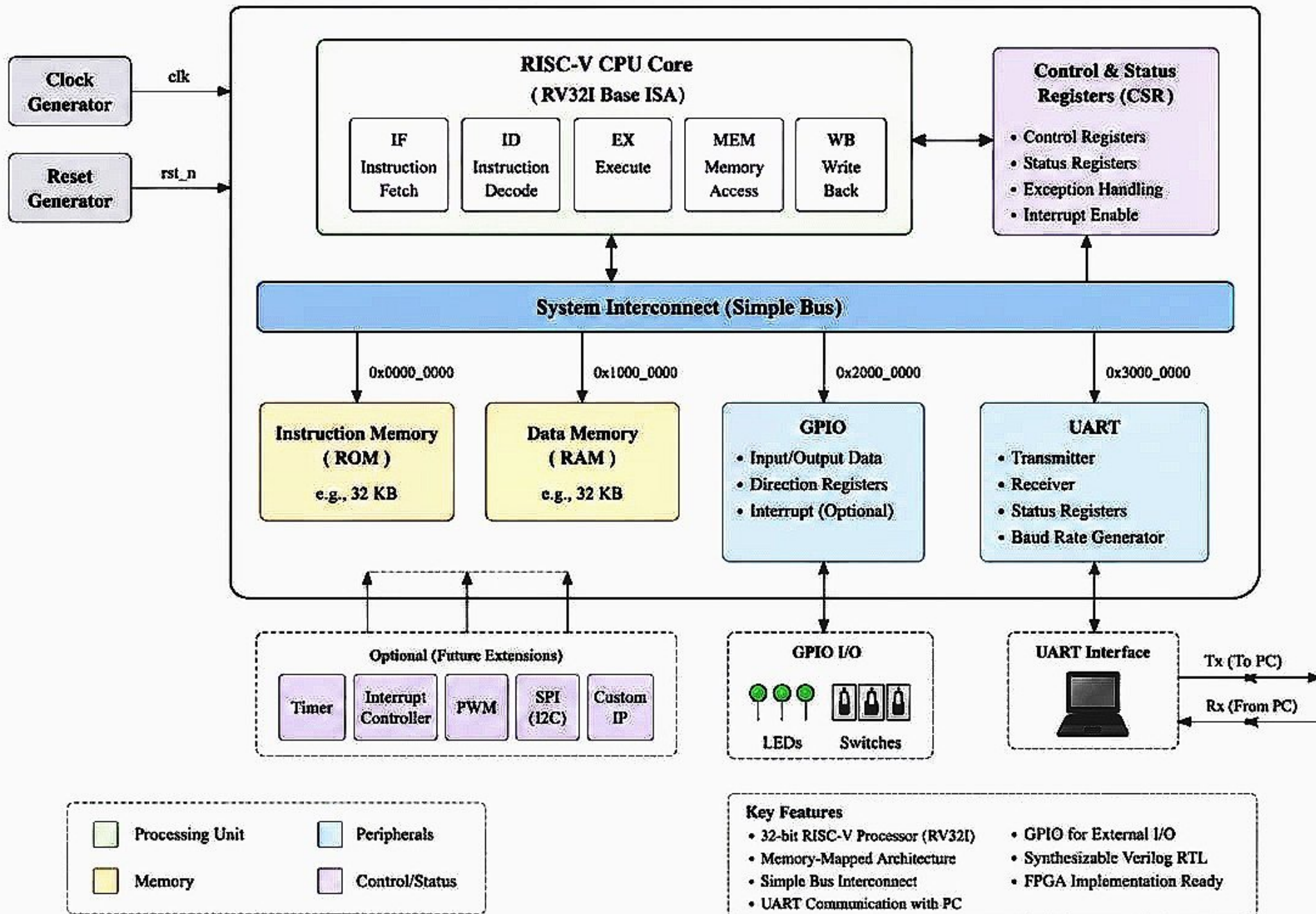




RISC-V SoC Architecture



Work Distribution & Technical Design Specification

Single-Cycle RV32I RISC-V Processor – FPGA Final Project

1. Supported ISA Subset (RV32I, restricted)

Instr	Type	Opcode	Operation
ADD	R	0110011	$rd = rs1 + rs2$ (funct3=000, funct7=0000000)
SUB	R	0110011	$rd = rs1 - rs2$ (funct3=000, funct7=0100000)
AND/OR/XOR	R	0110011	bitwise (funct3=111/110/100)
SLT	R	0110011	$rd = (rs1 < rs2) ? 1 : 0$ (funct3=010)
SLL/SRL/SRA	R	0110011	shifts (funct3=001/101, funct7 selects logical/arith)
ADDI	I	0010011	$rd = rs1 + imm$ (funct3=000)
LW	I	0000011	$rd = MEM[rs1 + imm]$ (funct3=010)
JALR	I	1100111	$rd = PC + 4; PC = (rs1 + imm) \& \sim 1$
SW	S	0100011	$MEM[rs1 + imm] = rs2$ (funct3=010)
BEQ	B	1100011	if ($rs1 == rs2$) $PC += imm$ (funct3=000)
BNE	B	1100011	if ($rs1 \neq rs2$) $PC += imm$ (funct3=001)
LUI	U	0110111	$rd = imm \ll 12$
JAL	J	1101111	$rd = PC + 4; PC += imm$

2. Module Interface Specifications

Every member should implement their module(s) exactly to these port lists first, so integration on Day 4 is plug-and-play.

2.1 Program Counter (PC) – Member 4

Signal	Dir	Width	Description
clk	in	1	clock
rst	in	1	synchronous reset
pc_next	in	32	next PC value (from MUX)
pc_out	out	32	current PC

2.2 Instruction Memory – Member 3

Signal	Dir	Width	Description
addr	in	32	byte address (use addr[7:2] to index)
instr	out	32	fetches instruction

2.3 Control Unit – Member 1

Signal	Dir	Width	Description
opcode	in	7	instr[6:0]

RegWrite	out	1	enable register file write
MemWrite	out	1	enable data memory write
MemRead	out	1	enable data memory read
MemtoReg	out	1	0:ALU result, 1:memory data
ALUSrc	out	1	0:rs2, 1:immediate
Branch	out	1	is a branch instruction
Jump	out	1	is a jump instruction (JAL/JALR)
ALUOp	out	2	coarse op class, feeds ALU Control

2.4 Register File – Member 2

Signal	Dir	Width	Description
clk	in	1	clock
rs1_addr	in	5	read port 1 address
rs2_addr	in	5	read port 2 address
rd_addr	in	5	write address
rd_data	in	32	write-back data
RegWrite	in	1	write enable
rs1_data	out	32	read data 1
rs2_data	out	32	read data 2

Note: x0 (address 0) must always read as 32'b0, regardless of writes.

2.5 Immediate Generator – Member 3

Signal	Dir	Width	Description
instr	in	32	full instruction
imm_out	out	32	sign-extended immediate

2.6 ALU – Member 2

Signal	Dir	Width	Description
a	in	32	operand 1 (rs1_data)
b	in	32	operand 2 (rs2_data or imm)
alu_ctrl	in	4	operation select (from ALU Control)
result	out	32	ALU result
zero	out	1	1 if result == 0 (for branches)

2.7 ALU Control – Member 1

Signal	Dir	Width	Description
ALUOp	in	2	from Control Unit
funct3	in	3	instr[14:12]
funct7_5	in	1	instr[30]
alu_ctrl	out	4	exact ALU operation code

2.8 Data Memory – Member 3

Signal	Dir	Width	Description
clk	in	1	clock
addr	in	32	byte address
write_data	in	32	data to store (SW)
MemWrite	in	1	write enable
MemRead	in	1	read enable
read_data	out	32	loaded data (LW)

2.9 Top-Level Datapath – Member 4

Instantiates and wires all modules above, plus the ALUSrc/MemtoReg/PC-source MUXes and the branch-target adder/comparator. Owns the overall clk/rst ports and the memory-mapped I/O output port for the demo.

2.10 I/O Interface – Member 5

Signal	Dir	Width	Description
reg_data	in	32	value of a designated register or memory address to display
leds	out	8	or 7-seg segments, mapped to reg_data[7:0]

3. Detailed Task Ownership

Member 1 – Control Unit & ALU Control

- Design the opcode → control-signal truth table (Section 1 above) and implement as Verilog case statement.
- Implement ALU Control mapping ALUOp+funct3+funct7[5] → 4-bit alu_ctrl.
- Write a standalone testbench applying every opcode and checking all control outputs.

Member 2 – ALU & Register File

- Implement ALU with all required operations (Section 1) and the zero flag.
- Implement 32× 32 register file, x0 hardwired to 0, synchronous write/ combinational read.
- Testbench: write/read every register, verify x0 stays 0, verify each ALU op with edge cases (overflow, equal operands, negative numbers).

Member 3 – Immediate Generator & Memories

- Implement immediate generation for I/S/B/U/J formats with correct sign-extension.
- Implement instruction memory (ROM, \$readmemh) and data memory (RAM, synchronous write).
- Prepare the hex machine-code file for the test assembly programs (hand-assemble or use a RISC-V assembler/simulator to generate it).

Member 4 – Top-Level Integration, PC, MUXes, Branch Logic

- Implement PC register, PC+4 adder, branch-target adder, and the PC-source / ALUSrc / MemtoReg MUXes.
- Implement branch comparator logic (BEQ/BNE) combined with Branch control signal.
- Integrate all members' modules into the top-level datapath on Day 4; own the reset/clock strategy and any timing fixes.

Member 5 – Testbench, Assembly Programs, I/O, Documentation

- Write 3 assembly test programs: (1) arithmetic sequence, (2) counting loop using BNE, (3) memory store/load round-trip.
- Write the top-level integration testbench that runs each program and checks final register/memory state.
- Implement the I/O interface module for the FPGA demo (LEDs/7-segment).
- Own the final report and presentation slides; coordinate the Day 5 demo script.

4. Suggested Assembly Test Programs

Program 1 – Arithmetic

```
addi x1, x0, 5    # x1 = 5
addi x2, x0, 10   # x2 = 10
add  x3, x1, x2    # x3 = 15
sub  x4, x2, x1    # x4 = 5
```

Program 2 – Loop with Branch

```
addi x5, x0, 0    # counter = 0
addi x6, x0, 5    # limit = 5
loop:
addi x5, x5, 1
bne  x5, x6, loop
```

Program 3 – Memory Store/Load

```
addi x7, x0, 100  # value = 100
sw   x7, 0(x0)    # MEM[0] = 100
lw   x8, 0(x0)    # x8 = MEM[0] = 100
```

5. Integration Checklist (Day 4)

- All individual module testbenches pass.
- Top-level module wires match the port tables in Section 2 exactly (no signal name/width mismatches).
- Run Program 1, 2, and 3; check final register values via waveform or \$display.
- Fix any control-signal or timing mismatches before moving to synthesis.

6. Risk & Time-Saving Notes

- Do NOT attempt pipelining or the full RV32I set — single-cycle + restricted subset is what makes the 5-day deadline realistic.
- Agree on port names/widths (Section 2) before anyone writes code — this is what prevents Day 4 integration delays.
- Keep instruction/data memory small (e.g. 64 words) to simplify indexing and \$readmemh loading.
- If FPGA synthesis time runs short on Day 5, simulation waveforms + a partial hardware demo are still an acceptable fallback — prioritize having a fully verified simulation first.